Blockchain-Based Library Book Lending Tracker for ALU

Oct 17, 2026 · @AMan

Course	Introduction to Blockchain Development (Software Engineering, Low-Level Specialization)
Assessment	Final Summative Project
Institution	African Leadership University (ALU), 2026 September Term
Student	Aman Abraha Kasa
Source code	github.com/Aman-Kasa/blockchain-coursework/summative_project
Demo video	youtu.be/ZojWApYIyjM

Abstract

The system is a complete blockchain-based lending ledger for the ALU library, written in C11 with OpenSSL 3. Every borrow, return, reservation, renewal, overdue notice, fine payment and token transfer becomes a record signed with a real ECDSA P-256 key, waits in a prioritised pending pool, and becomes final only when a miner seals it into a proof-of-work block. Blocks carry a Merkle root built from scratch, and the whole state is persisted to disk and re-validated on every start.

The implementation compiles with `-Wall -Wextra -Wpedantic` and zero warnings, and passes an automated suite of 91 checks, including an AddressSanitizer and UndefinedBehaviorSanitizer run.

1. Introduction and problem statement

Traditional library records can be changed quietly by whoever controls them. A librarian can mark a lost book as returned, a borrower can dispute what a paper log shows, and a due date can be altered after the fact. Nobody can prove afterwards what the original record said.

This project replaces the editable log with a hash-linked, signed ledger. Its design goals are:

1. **Authenticity:** every lending action is signed by the member (or the librarian node) it concerns, so nobody can act in someone else's name.
2. **Immutability:** a confirmed record cannot be changed without breaking its signature, its Merkle root, its block hash and every later link.
3. **Accountability:** fines, rewards and copy states are derived from the chain, never typed in directly, so the ledger is the single source of truth.
4. **Misuse resistance:** suspicious patterns are held for a librarian's review instead of being mined automatically.

The work combines Formative 1 (registries, SHA-256 hash chain, signatures, validation and CLI) and Formative 2 (token rewards, pending pool, mining, UTXO and account ledgers), and adds every summative extension in the brief: copy-level inventory, reservations, renewals, fines, real ECDSA, nonce replay protection, fraud heuristics and disk persistence.

2. System architecture

The program is split into 17 modules of about 4,600 lines in total, each with one job. All state that matters is derived from the chain, so every other structure can be rebuilt from it.

Module	Responsibility
`crypto.c`	SHA-256, ECDSA P-256 key generation, signing and verification (OpenSSL EVP)
`registry.c`	Loads and validates `books.txt`, `members.txt` and `copies.txt`
`record.c`	Lending record layout, canonical content, signing and hashing
`merkle.c`	Merkle tree construction and printing, written from scratch
`chain.c`	Blocks, proof of work, linking and full chain validation
`state.c`	Copy state machine, reservation queues, nonces and fines
`lending.c`	Borrow, return, reserve, renew, overdue and fine commands
`pool.c`	Pending pool, priority ordering and projected state

Module	Responsibility
`fraud.c`	The four misuse heuristics
`utxo.c`, `account.c`, `ledger.c`	The two token models, rebuilt from the chain
`mining.c`	Solo, pool and cloud mining
`persist.c`	Saving and restoring `chain.dat`, `state.dat` and `pool.dat`
`cli.c`, `main.c`	Command-line interface and start-up

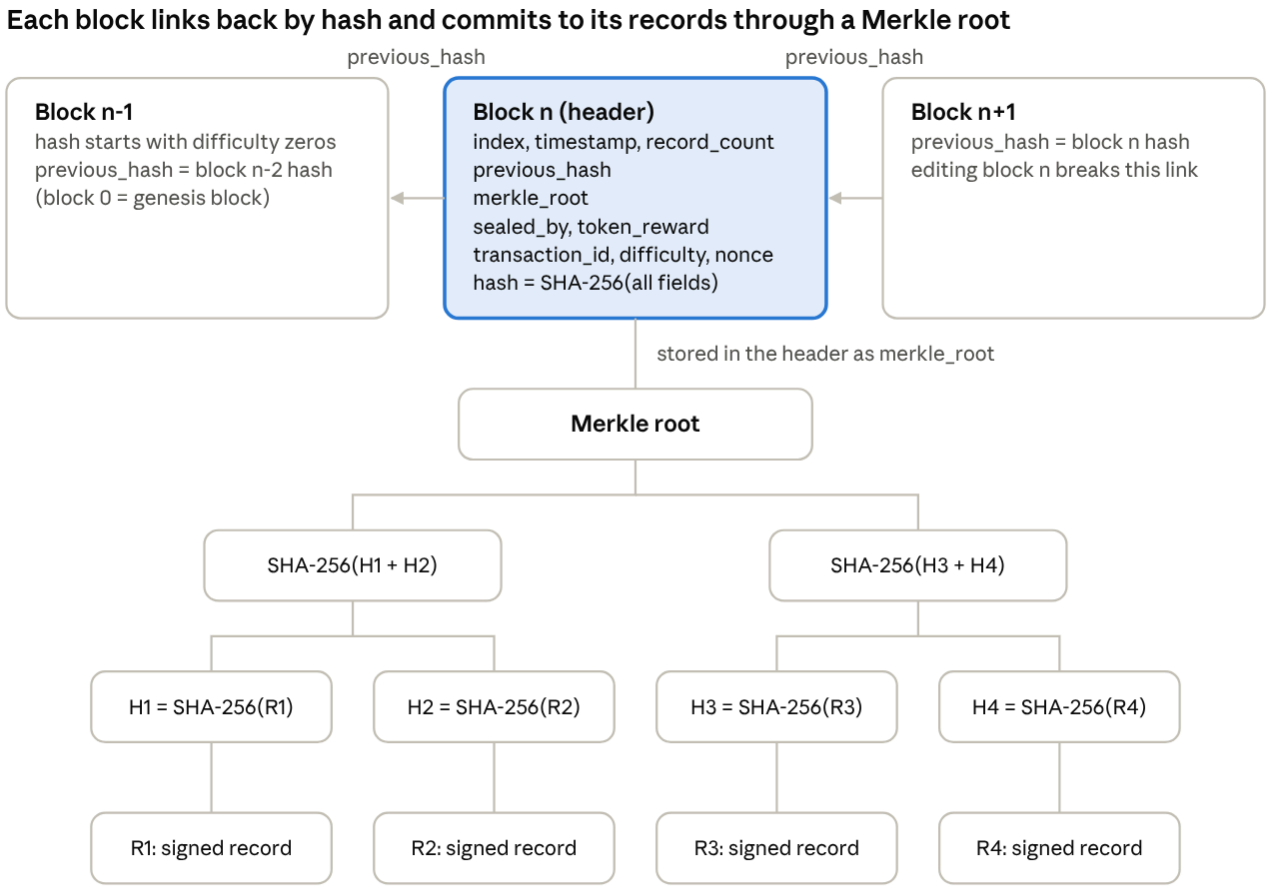


Diagram 1 · blockchain structure with Merkle tree

A record is final only once it sits inside a mined block. Changing any record alters its leaf hash, the Merkle root, the block hash and every later `previous_hash`.

3. Blockchain data structures

The chain is a singly linked list of heap-allocated `Block` structs. Each block owns an array of `Record` structs, so a block can hold several lending records.

Block header

Field	Type	Meaning
`index`	`int`	Position in the chain; 0 is the genesis block
`timestamp`	`time_t`	Creation time, strictly greater than the previous block's
`record_count`	`int`	Number of records in the block
`previous_hash`	`char[65]`	SHA-256 hash of the previous block
`merkle_root`	`char[65]`	Root of the Merkle tree over the record hashes
`hash`	`char[65]`	SHA-256 over all header fields
`sealed_by`	`char[32]`	Miner or node that mined the block, such as `solo:LIB-NODE-1`
`token_reward`	`int`	Sum of member rewards in the block (10 on time, 5 late, 0 none)
`transaction_id`	`char[65]`	SHA-256 over the block's reward transaction IDs
`difficulty`, `nonce`	`int`, `unsigned long`	Proof-of-work target and the nonce that met it

Storing the difficulty in each block lets blocks mined at different settings all validate.

Lending record

Each `Record` contains every field the brief requires: `record_id`, `book_id`, `copy_id`, `book_title`, `member_id`, `member_name`, `action`, `due_date`, `fine_amount`, `nonce` and `signature`. Titles and names are copied from the registries when the record is created, so the chain keeps the values as they were at that moment.

Four more fields support the extensions: `token_reward`, `condition` and `reason` (for returns and librarian overrides), and `signer_id` with `pubkey` (the signer's identity). The action set is the brief's five (`BORROWED`, `RETURNED`, `RESERVED`, `RENEWED`, `OVERDUE`) plus three the system needs on the chain: `LOST`, `FINE_PAID` and `TRANSFER`.

The `record_id` is the first 16 hex characters of SHA-256 over the record's canonical content. That makes it a fingerprint: edit the record and its ID no longer matches.

Pending pool entry

A `PoolEntry` wraps a fully signed `Record` (whose `record_id` is the `request_id`) with `priority` (0 for overdue-related, 1 otherwise), `sort_date` (the reservation or submission date), `status` (`PENDING`, `CONFIRMED` or `SUSPICIOUS`) and a `flag` explaining why it was held. The `book_id`, `copy_id`, `member_id` and `action_requested` fields come from the wrapped record.

Registries

`books.txt` (`book_id,title,author`) and `members.txt` (`member_id,full_name,course_code`) keep the Formative 1 format unchanged. `copies.txt` (`copy_id,book_id,condition`) adds the physical copies. At start-up the loader checks that each file exists and is not empty, and that every copy refers to a registered book. Every lending command then checks that its IDs exist before anything is signed, which enforces referential integrity between the registries and the chain.

4. Merkle tree construction and verification

The Merkle tree is implemented from scratch in `merkle.c`, with no external Merkle library. It turns a block's records into one 32-byte root that changes if any record changes.

Construction (`merkle_root`):

1. Each leaf is `record_hash(r)`: SHA-256 over the record's canonical content **and** its signature, so a swapped signature also changes the leaf.
2. Leaves are paired left to right. Each parent is SHA-256 over the two child hashes as raw bytes (32 + 32 = 64 bytes), not as hex text.
3. If a level has an odd number of nodes, the last one is paired with itself, as in Bitcoin.
4. Pairing repeats, level by level, until one hash remains: the root. A block with no records uses a root of 64 zeros.

Where the root is used:

- **Storage:** `block_prepare` computes the root once and writes it into the header field `merkle_root`, which is then covered by the block hash and the proof of work.
- **Verification:** `validate_chain` recomputes the root from the stored records and compares it with `merkle_root`. A mismatch marks the block as tampered.
- **Inspection:** `merkle_view <index>` prints every level of a block's tree, including which node was paired with itself.

The root makes record-level tampering visible in the header. Even an attacker who also recomputes the root and re-mines the block is still stopped by the ECDSA signatures, as Section 12 shows.

5. Copy inventory, reservations and fines

Availability is tracked per physical copy, so a title with two copies can be partly available. Each `CopyState` holds `copy_id`, `book_id`, `condition` (`NEW`, `GOOD`, `WORN`, `LOST`), `status` (`AVAILABLE`, `BORROWED`, `RESERVED`, `LOST`), the current holder, the due date, the renewals used and an overdue flag.

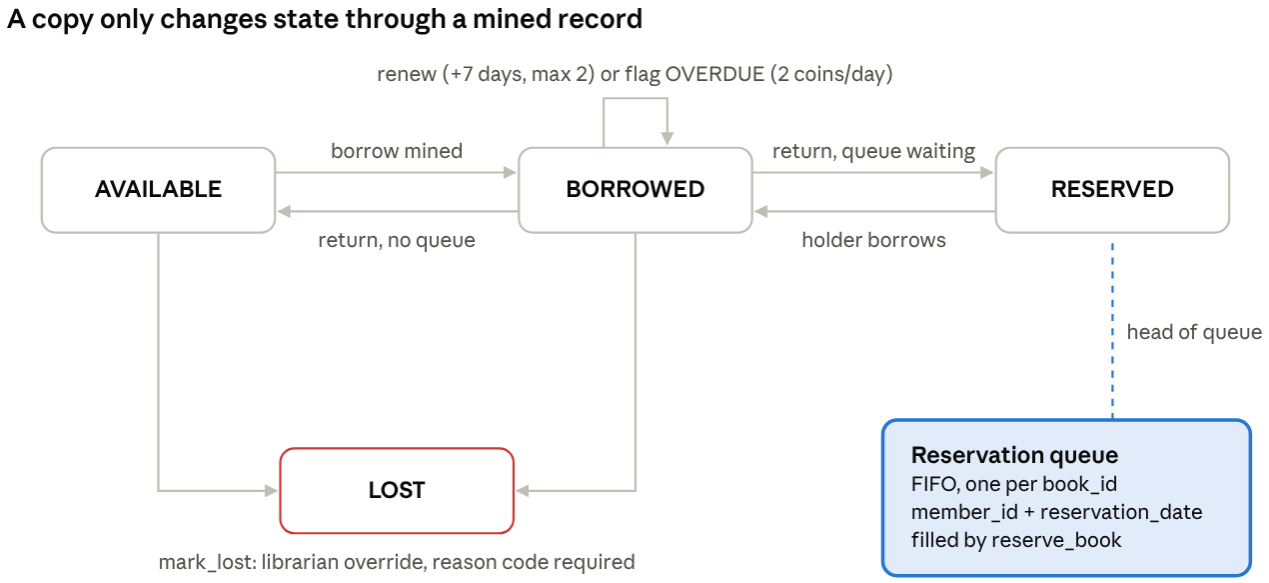


Diagram 3 · copy inventory state machine and reservation queue

The state machine is enforced in `state.c` when a record is applied. Any other transition is rejected, both when a request is submitted and again when the chain is replayed during validation.

Reservation queue

Each title has a first-in-first-out queue of `member_id` and `reservation_date` entries. `reserve_book` is accepted only when no copy of that title is available. When a copy is returned and the queue is not empty, the copy moves to `RESERVED` for the member at the head of the queue instead of becoming `AVAILABLE`. Only that member can then borrow it.

Due dates, renewals and fines

Rule	Value (`config.h`)
Loan period	14 days from the borrow
Renewal	+7 days, at most 2 per loan, refused if another member has reserved the title
Fine	2 coins per full day late
Return reward	10 coins on time, 5 late, none while still overdue

On every command, the overdue scan checks all borrowed copies. A copy past its due date gets a librarian-signed `OVERDUE` record with high priority, and its fine keeps accruing. On a late return, the `RETURNED` record carries the final `fine_amount`, which is added to the member's fines owed. `settle_fine` creates a signed `FINE_PAID` record, so payments are on the chain too. A simulated clock (`advance_time <days>`) makes these rules testable without waiting real days.

6. Pending pool and the token reward flow

No command changes the library directly. Every lending action, fine payment and token transfer first becomes a signed request in the pending pool, and takes effect only when it is mined.

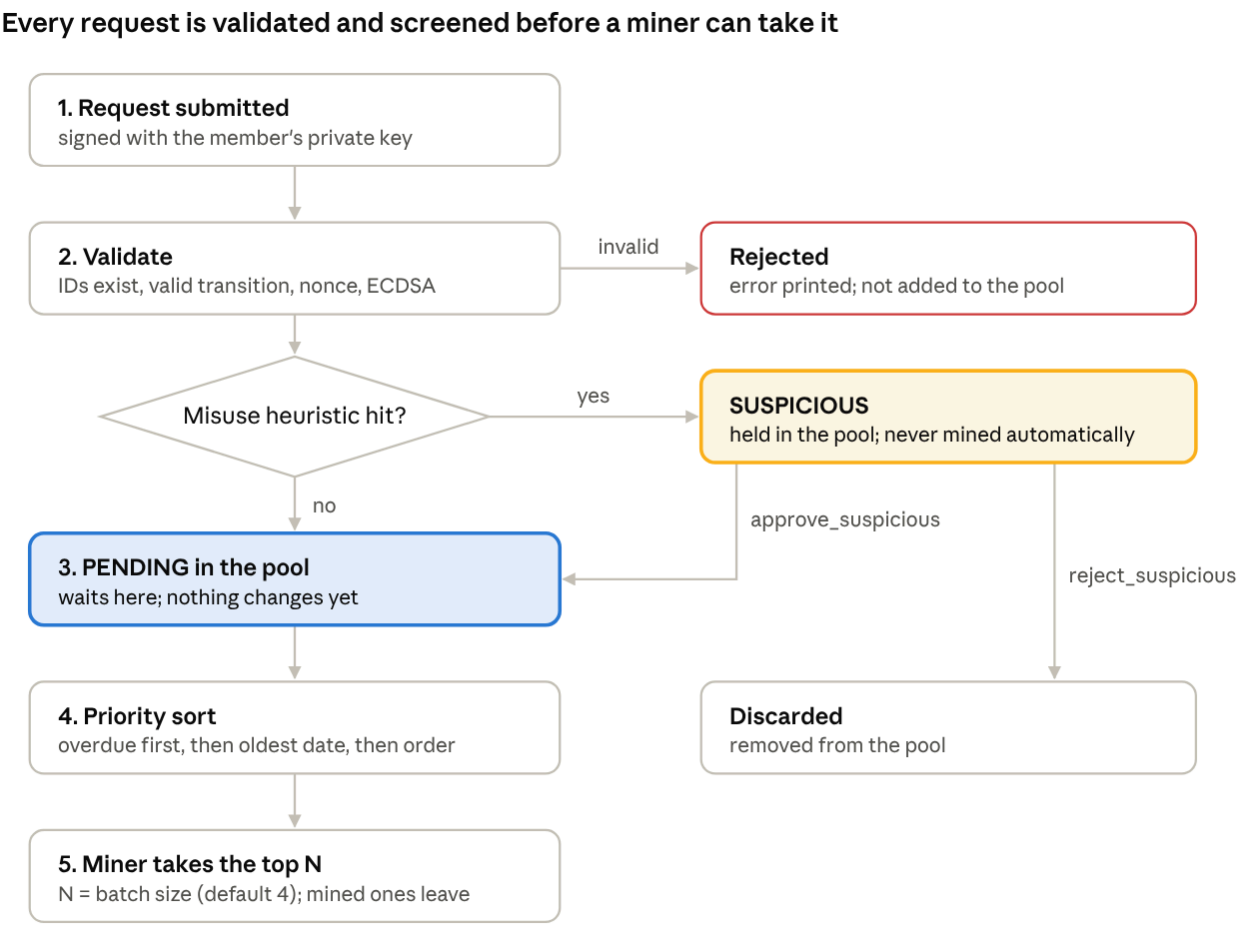


Diagram 2 · pending pool flow with prioritisation and fraud flagging

Validation on entry

A new request is checked against a **projected state**: the confirmed state with every pending request applied in submission order. This stops two pending requests from claiming the same copy, and it gives the member's expected next nonce. Requests with unknown IDs, an invalid state transition, a wrong nonce or a failing signature are rejected and never enter the pool.

Prioritisation

`pool_priority_order` sorts the pool by three keys, as the brief requires:

1. `priority`: overdue-related requests (0) before everything else (1).
2. `sort_date` ascending: the member who reserved earliest, or submitted earliest, goes first.
3. Submission order, as a tie-breaker.

The miner always takes the top N `PENDING` entries in this order. `SUSPICIOUS` entries are skipped.

Batching

Blocks use **fixed-size batching**: N records per block, 4 by default, adjustable from 1 to 16 with `--batch` or `batch_size`. If fewer than N requests are waiting, the block takes all of them. Fixed-size batching was chosen over a time window because it is deterministic, which keeps tests and the demo repeatable. After mining, the confirmed entries are removed from the pool.

From return to balance

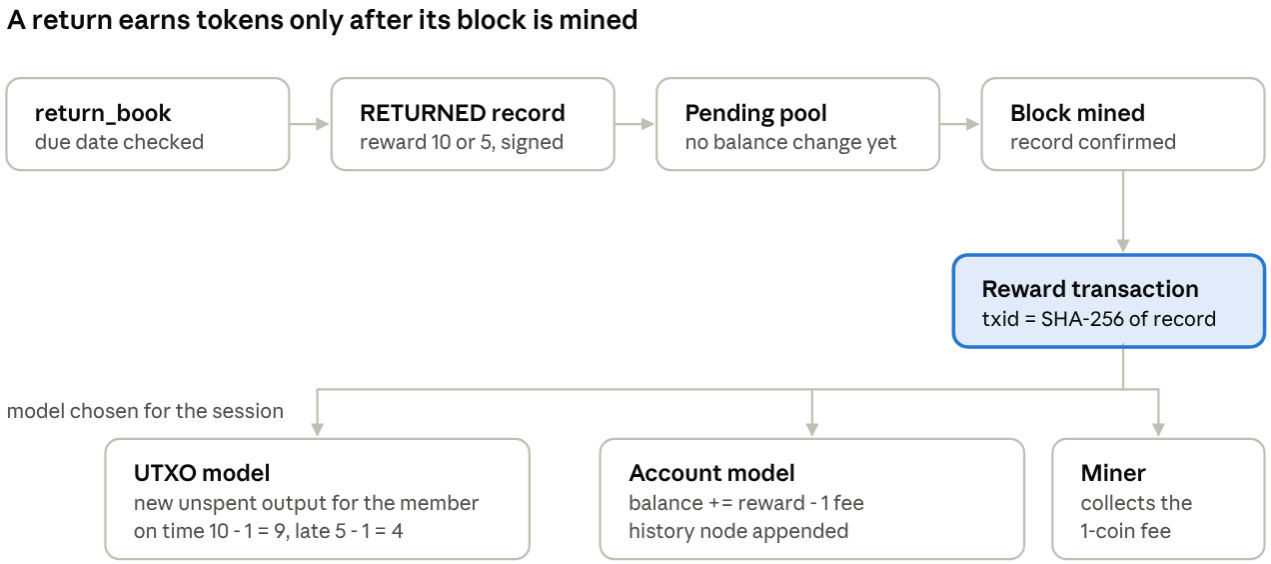


Diagram 4 · token reward flow from return to balance update

When `return_book` runs, the system compares the return time with the due date and sets `token_reward` to 10 (on time) or 5 (late), plus `fine_amount` for a late return. The member's balance does not change while the request waits. Once the block is mined, the reward becomes a transaction whose ID is a SHA-256 hash of the record. A fixed 1-coin fee is deducted for the miner, so the member receives 9 or 4 coins. A copy that stays overdue and is never returned creates no reward.

7. Token ledger models

Both models use registry member IDs as account identifiers, start every member at 0, and earn tokens only through mined returns. The model is chosen once per session with `--model utxo` or `--model account`, or at the start-up prompt. Balances are rebuilt from the chain on every start (`ledger_rebuild`), so both models work on the same chain and can be switched between sessions.

UTXO model (`utxo.c`)

Coins are unspent transaction outputs, kept as a linked list of `Utxo` nodes with `txid`, `vout`, `owner`, `amount`, `spent` and `spent_by`. A member's balance is the sum of their unspent outputs.

- **Rewards** are transactions with no inputs, like a coinbase. They create one output of `reward - 1` for the member.
- **Transfers** select the sender's oldest unspent outputs until they cover `amount + 1` fee. Those inputs are marked spent, one output pays the recipient, and any excess returns to the sender as a **change output**.
- **Rejection:** if the inputs cannot cover the amount plus the fee, the transfer is rejected.
- **Double spending** is impossible because a spent output is flagged with the transaction that spent it and is never selected again.
- The full UTXO set (ID, owner, amount, creating `txid:vout`) is printed after every confirmed block, and with `utxo_view`.

Account model (`account.c`)

Each member is an `Account` with a `balance`, a `nonce` and a transaction history stored as a linked list of `TxRecord` nodes (`sender`, `recipient`, `amount`, `fee`, `nonce`, `timestamp`).

- **Rewards** credit `reward - 1` to the balance directly after the block is confirmed.
- **Transfers** (`account_transfer <from> <to> <amount> <nonce>`) debit `amount + 1` from the sender and credit `amount` to the recipient. A transfer is rejected if the sender's balance cannot cover it.
- **Replay protection:** the nonce must equal the account's next nonce. A wrong or reused nonce is rejected, and the nonce advances with every outgoing transaction.
- `transaction_history <member_id>` walks the linked list and prints every transaction for that member.

Comparison

Aspect	UTXO	Account
What is stored	Individual coins (outputs)	One balance per member
Balance	Computed by summing unspent outputs	Read directly
Double-spend defence	Each output can be spent only once	Balance check plus a strict nonce

Aspect	UTXO	Account
Change	Explicit change output to the sender	Not needed
Replay defence	Spent inputs cannot be reused	Nonce must match exactly
Auditability	Every coin traces back to the reward that created it	Needs the history list to explain a balance
Simplicity	More bookkeeping	Simpler to read and update
Used by	Bitcoin	Ethereum

UTXO gives stronger, coin-level auditability, which suits a ledger whose purpose is accountability. The account model is simpler and makes balances and histories easier to read. Implementing both on the same chain shows that the choice affects bookkeeping, not the lending records themselves.

8. Mining simulation

Mining is the only way a request becomes confirmed. All three methods draw from the same pending pool, build the block the same way and use the same proof of work.

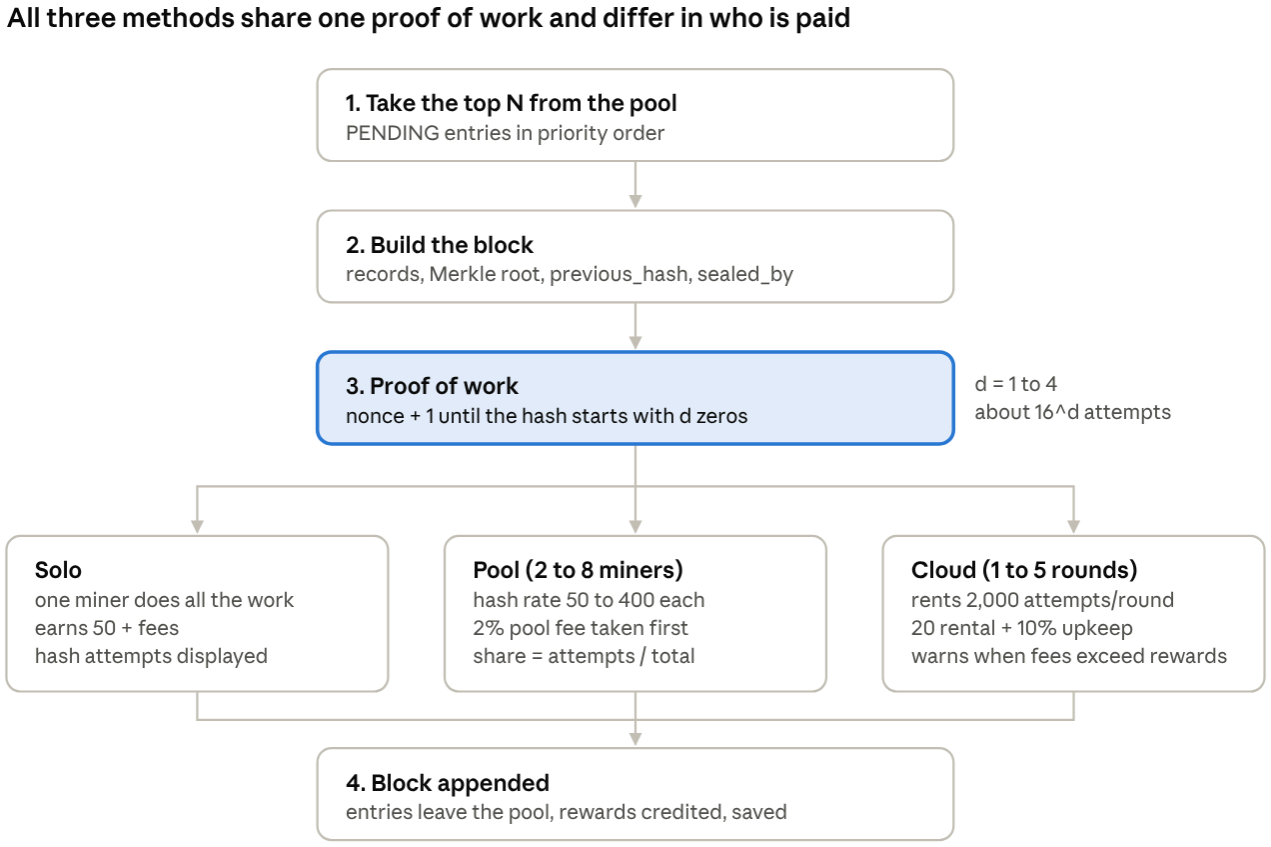


Diagram 5 · mining workflow with difficulty and the solo, pool and cloud paths

Proof of work and difficulty

The difficulty `d` is the number of leading zero hex characters the block hash must have. The miner increments the block's `nonce` and recomputes SHA-256 over the header until the target is met, then prints the number of attempts. Each extra zero multiplies the expected work by 16:

Difficulty	Target prefix	Expected attempts
1	`0`	16
2 (default)	`00`	256
3	`000`	4,096
4	`0000`	65,536

Difficulty is set with `--difficulty <1-4>` at start-up or `difficulty_status set <n>` during a session. Each block stores its own difficulty, so a change never invalidates older blocks.

The three methods

- **Solo (`mine_solo`):** one librarian node does all the hashing and receives the full 50-coin block reward plus the transaction fees, on top of any member rewards in the block.
- **Pool (`mine_pool [2-8]`):** each simulated miner gets a random hash rate of 50 to 400 attempts per round. A 2% pool fee is deducted from the total reward first, and each miner receives `(miner_attempts / total_attempts) x remaining reward`. A table shows miner ID, hash rate, attempts, share and reward, and marks the miner that found the block.
- **Cloud (`mine_cloud <1-5>`):** the user rents 2,000 attempts per round. Each round costs a 20-coin rental fee plus 10% maintenance on that round's rewards. The summary shows gross earnings, total fees and net profit, and a warning appears in any round where cumulative fees exceed cumulative rewards. In the demo, rounds 2 and 3 had nothing to mine, so the rental ended at a net loss of 14.10 coins and the warning was shown.

9. ECDSA signature scheme and replay protection

Every record is signed with real ECDSA on the NIST P-256 curve (`prime256v1`) through OpenSSL 3, replacing the placeholder signatures of the formatives. SHA-256 is also provided by OpenSSL libcrypto.

Key generation

`crypto_identity_load_or_create` runs for every member at start-up and when `register_member` adds one. If `keys/<member_id>.pem` exists, the private key is loaded; otherwise `EVP_EC_gen("prime256v1")` creates a new key pair. The file's permissions are set to owner-only (0600) before any key material is written. The librarian node has its own key, `keys/LIBRARIAN.pem`, used for system records such as `OVERDUE`.

The public key is exported and compressed to 33 bytes (a `02` or `03` prefix plus the X coordinate), giving a 66-character hex string. That string is the member's on-chain identity, stored in every record they sign.

Signing

1. `record_content` writes every field except `record_id` and the signature in a fixed order, separated by `|`. This canonical form means the same record always produces the same bytes.
2. `record_id` is set to the first 16 hex characters of SHA-256 over that content.
3. `EVP_DigestSign` with SHA-256 signs the content using the signer's private key, producing a DER-encoded signature stored in the record.

Verification

`record_verify` recomputes the `record_id`, then calls `EVP_DigestVerify` with the public key from the record. A record is verified at three points: when it enters the pool, before a miner includes it in a block, and for every record during `validate_chain`. Validation also checks that the record's public key matches the signer's registered key, so an attacker cannot sign with a key of their own. A record with an invalid or missing signature is rejected.

A note on the OpenSSL API

The brief mentions OpenSSL's `EC_KEY` API. That API is deprecated in OpenSSL 3.0, so this project uses its replacement, the high-level EVP interface (`EVP_EC_gen`, `EVP_DigestSign`, `EVP_DigestVerify`). The algorithm, ECDSA over P-256 with SHA-256, is the same, and the code compiles without deprecation warnings.

Nonce-based replay protection

Every signer has a nonce starting at 0, and the nonce is part of the signed content. A request is accepted only if its nonce equals the signer's expected next nonce, and the confirmed nonce advances only when the record is mined. The `replay_request <id>`

command simulates an attacker resubmitting a captured signed request byte for byte. Because its `request_id` already exists, it is flagged `SUSPICIOUS` as a duplicate and is never mined automatically; even if approved, its used nonce would fail.

10. Misuse detection and the review workflow

`fraud_check` in `fraud.c` runs on every new request, against the state before the request applies. A hit does not reject the request. Instead the request enters the pool as `SUSPICIOUS`, with the reason saved for audit.

Heuristic	Triggers when	Example reason printed
Excessive concurrent borrows	The member would hold more than 3 copies at once	`ALU004 already holds 3 copies (maximum 3)`
Rapid re-borrow	The member re-borrows a title within 24 hours of returning it	`ALU001 returned BK001 2h 05m ago (within 24 hours)`
Duplicate request	The `request_id` already exists in the chain or the pool	`... is already in the pending pool`
Condition manipulation	A return downgrades the copy more than one level (e.g. `NEW` to `WORN`) without a librarian reason code	`BK001-C1 downgraded NEW -> WORN (2 levels)`

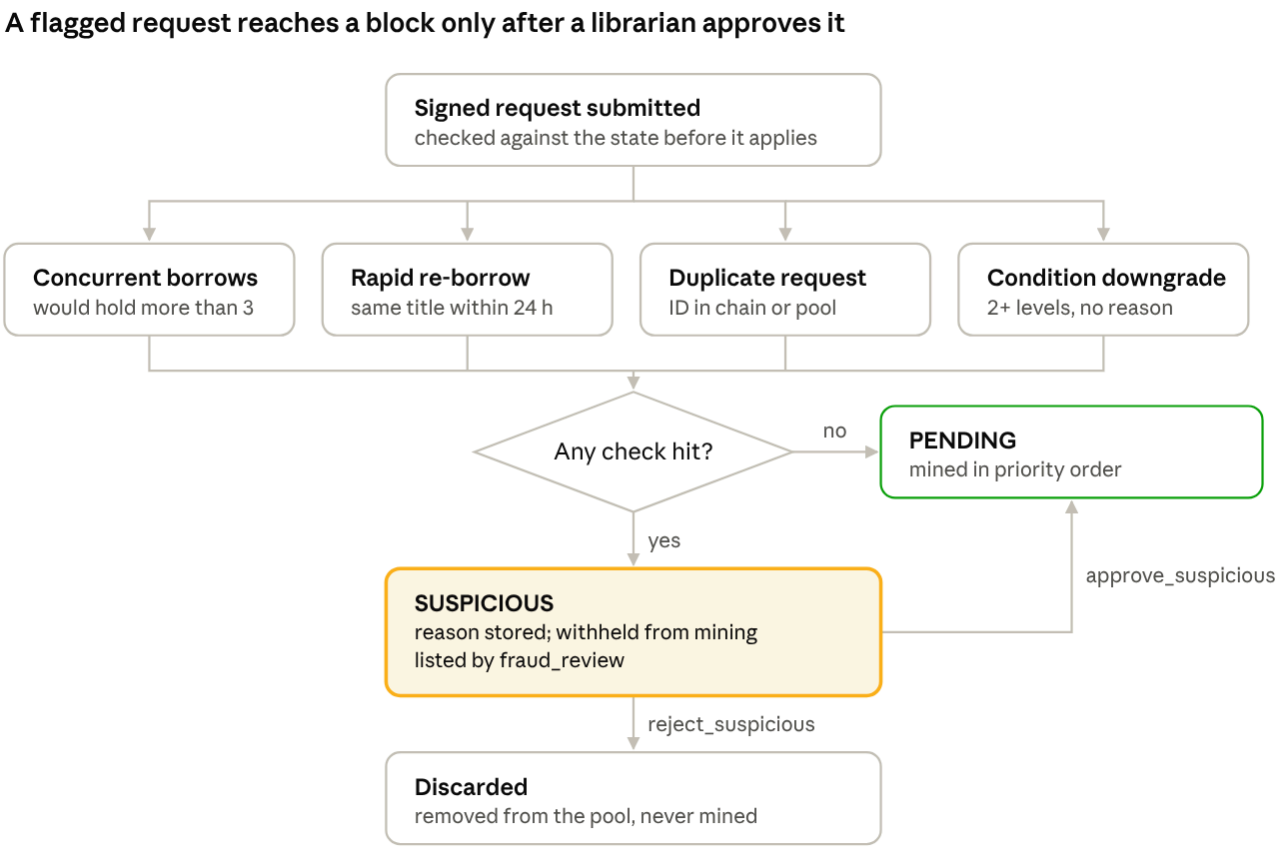


Diagram 6 · fraud detection pipeline from submission to review

The review workflow has three commands. `fraud_review` lists every suspicious request with its signer, nonce and reason. `approve_suspicious <id>` changes it to `PENDING`, so a miner can include it. `reject_suspicious <id>` removes it from the pool. While a request is `SUSPICIOUS`, the block builder skips it, so it can never be mined automatically. In testing (Scenario 4, Section 13), ALU004's fourth borrow was flagged and rejected, and the next block contained only the four legitimate requests.

11. Disk persistence

The full system state survives restarts. It is written as **plain-text structured `.dat` files**, one record per line with `|`-separated fields and a type tag at the start of each line. Plain text was chosen over binary because it is portable across compilers and struct layouts, readable when debugging, and easy to edit for tamper-detection tests.

File	Line types	Contents
`data/chain.dat`	`B|...` then its `R|...` lines	Every block header (`index` , `timestamp` , `record_count` , `previous_hash` , `merkle_root` , `hash` , `sealed_by` , `token_reward` , `transaction_id` , `difficulty` , `nonce`), each followed by its records with their signatures in hex
`data/state.dat`	`META` , `COPY` , `QUEUE` , `SIGNER` , `RETURN` , `UTXO` or `ACCOUNT` , `MINER`	Difficulty, batch size, block reward, clock offset, ledger model, chain length and tip hash; copy inventory; reservation queues; nonces and fines; recent returns; the UTXO set or account balances; miner earnings
`data/pool.dat`	one line per entry	Every pending or suspicious request, fully signed, with its priority, date, status and flag

When it saves: after every command that changes state, after each mined block and on exit (also on demand with `chain_save`). Each file is written to a temporary file first and then renamed, so a crash mid-write cannot leave a half-written file.

When it loads: at start-up the program checks for `chain.dat` . If none exists, it creates a new chain with a genesis block. If it exists, the program:

1. Parses the chain and runs the full `validate_chain` check before any command is allowed.
2. Replays every record from the genesis block to rebuild the inventory, queues, nonces, fines and balances.
3. Compares that rebuilt state with `state.dat` . If they differ, the saved file is reported as not matching the chain and the chain's version is used.
4. Reloads `pool.dat` , re-verifying each signature and discarding any entry that fails.

If validation fails, the session becomes **read-only**: lending, mining and transfers are disabled, but `validate_chain` , `chain_view` and `merkle_view` still work so the damage can be inspected. The chain is the source of truth, so `state.dat` and `pool.dat` can never override it.

12. Chain validation and tamper detection

`validate_chain` checks every block from the genesis block onwards and stops at the first problem, reporting the block and the reason. For each block it checks:

1. The index is in order and `previous_hash` equals the previous block's stored hash.
2. The timestamp is strictly greater than the previous block's.
3. Every record's `record_id` matches its content, its ECDSA signature verifies, and its public key matches the signer's registered key.
4. The Merkle root recomputed from the records equals `merkle_root`.
5. `token_reward` and `transaction_id` match the rewards in the block's records.
6. The header hash recomputed from all fields equals the stored `hash`, and that hash meets the block's proof-of-work difficulty.

After the structural checks, every record is **replayed through the state machine**: nonces, copy transitions, fines and rewards must all be valid in order. A chain that is well-formed but describes an impossible history, such as returning a copy that was never borrowed, is rejected too.

Why the signatures matter most

Hashes, Merkle roots and proof of work alone do not make a private chain tamper-proof, because whoever holds the file can recompute them all. The test script `tests/forge_chain.py` proves this. It extends a due date by 30 days, then does everything an attacker without the member's private key can do: recompute the record ID, the Merkle root and the reward ID, re-mine the block and re-link and re-mine every later block. The forged chain passes every hash and proof-of-work check, but `validate_chain` still reports `ID or ECDSA signature is invalid`, because the attacker cannot produce a valid signature for the changed record.

This directly addresses the brief's motivating fraud: a due date silently altered after the fact is detected, and the session falls back to read-only.

13. Testing results

All six documented scenarios behaved as expected, and the automated suite passed 91 of 91 checks. The scenarios were run on a fresh chain with `./library_chain --model utxo --difficulty 2 --seed 7` and the sample registries; the actual output is quoted from the program.

#	Scenario and commands	Expected	Actual output	Result
1	Invalid IDs: `borrow_book ALU999 BK001`, then `borrow_book ALU001 BK999`	Both rejected; nothing enters the pool	`ERROR: Book or Member not found` (twice); `pool_view` empty	Pass
2	Reservation and on-time return: ALU001 holds BK002-C1; `reserve_book ALU002 BK002`, mine; `return_book BK002-C1`, mine	Copy goes to ALU002, not back to AVAILABLE; ALU001 earns 10 - 1 fee = 9	`Reward transaction e4e2d089c3bf...` `confirmed: ALU001 credited 10 - fee 1 = 9 coins` and `Copy BK002-C1 is now RESERVED for ALU002`; UTXO set shows `#1 ALU001 9`	Pass
3	Late return with fine: ALU003 borrows BK001-C1, mine; `advance_time 20`; `check_overdue`; `return_book BK001-C1`, mine	6 days late (14-day loan); fine 6 x 2 = 12; reward 5 - 1 = 4; OVERDUE request has HIGH priority	`6 day(s) past due: fine so far 12 coins`, `priority HIGH: overdue-related`; after mining `ALU003 credited 5 - fee 1 = 4 coins` and `fine of 12 coins added`	Pass
4	Fraud flag: ALU004 already holds or has pending 3 copies; `borrow_book ALU004 BK001`	Flagged SUSPICIOUS and left out of the next block	`FLAGGED SUSPICIOUS ... excessive concurrent borrows: ALU004 already holds 3 copies (maximum 3)`; next block had 4 records, none of them this request	Pass
5	Pool mining: `mine_pool 4` on a block with no fees	2% fee on 50 = 1.00; 49.00 shared by attempts	`pool fee 2% = 1.00; distributed 49.00`; shares 24.44%, 9.63%, 29.63%, 36.30% summing to 100%	Pass

#	Scenario and commands	Expected	Actual output	Result
6	Tamper detection: change one due date in an `R|` line of `chain.dat`, restart, `validate_chain`	Chain rejected; session read-only	`chain validation FAILED at block 1: record 42d2ba434640a751: ID or ECDSA signature is invalid`; `Chain status: 12 block(s), COMPROMISED`	Pass

Automated test suite

`make test` runs `tests/run_tests.sh`, which builds the program and runs **91 checks: 91 passed, 0 failed**. The checks cover the core cryptography unit tests (ECDSA, Merkle, blocks), registries, lending and the pending pool, reservations, renewals, overdue detection and fines, the copy state machine, pool priority and multi-record blocks, nonces and replay protection, both ledgers, all three mining methods, fraud detection, persistence and tamper detection.

The tamper tests go beyond simple edits: a re-mined forgery (Section 12), an edited block header, an edited `state.dat` (detected and rebuilt from the chain) and an edited `pool.dat` entry (discarded for its bad signature). The suite ends with an AddressSanitizer and UndefinedBehaviorSanitizer build, which reported no memory errors, leaks or undefined behaviour in either ledger model.

14. Limitations, future work and conclusion

Limitations

- **Single node:** the chain runs on one machine. Signatures stop forged records, but someone with full control of the files and every private key could still rewrite history. A real deployment would need independent nodes.
- **Private keys on disk:** keys are stored unencrypted in `keys/`, protected only by file permissions. A production system would keep them in a password-protected keystore or hardware token.
- **Simulated mining and time:** proof of work is real SHA-256 hashing, but pool and cloud miners are simulated in one process, and overdue logic uses a simulated clock (`advance_time`) for testing.

- **Fixed capacities:** registries and state use fixed-size arrays (for example 512 copies and 128 members), which keeps state easy to copy and save but sets upper limits.

Future work

- Run the ledger as a small **permissioned network** in which the library, the registrar and an auditor each hold a node, so no single party can rewrite the chain.
- Encrypt private keys at rest and add key rotation.
- Add a web or mobile front end for members, using the same signed-request model.

Conclusion

The system meets every requirement in the brief: Formative 1 and 2 features, plus copy-level inventory, reservations, renewals, fines, real ECDSA signatures, nonce replay protection, four fraud heuristics with manual review, and disk persistence with full re-validation. It compiles cleanly with `-Wall -Wextra -Wpedantic`, and 91 automated checks pass. Most importantly, it solves the problem it set out to solve: once a lending record is confirmed, a librarian or borrower can no longer change it without the change being detected.